

Targeted Counterfactual Branching for Tool-Invocation Decisions in RL Fine-Tuning

Andres Garcia Joshua Hsieh Prakash Koukuntla

CS 224R, Stanford University — {andg, joshhsieh, prakashk}@stanford.edu

Extended Abstract

Motivation and problem statement. Tool-augmented reasoning is now standard in capable language models: they call calculators, code interpreters, and search constantly. The difficult part is *when* to issue one. Standard online RL post-training (e.g. RLOO) hands the model a single scalar reward for an entire trajectory. When a tool is available, the model makes a separate binary decision—call or skip—at many points in that trajectory, but RLOO smears one reward across hundreds of tokens, so the gradient on any one call/skip decision is diluted. Critically, plain RLOO can never ask the counterfactual that actually matters for this decision: *what would have happened if the model had skipped (or made) this particular call?*

Method and novelty. We propose *Targeted Counterfactual Branching*, an inner augmentation to RLOO that fires only at tool-invocation decision points. At each token, we compute p_{call} , the probability mass the policy places on tool-opening tokens, and the binary entropy $H(p_{\text{call}}, 1 - p_{\text{call}})$. When this entropy exceeds a threshold τ —i.e. the model is genuinely on the fence—we fork the rollout: a *call* branch forced to emit the tool token and a *skip* branch forced to emit the top non-tool token, both completed and scored by the same verifier. Because the two branches share an identical prefix and differ in exactly one token, the reward difference $A_{\text{call}} = R_{\text{call}} - R_{\text{skip}}$ is a clean, low-variance, paired estimate of the value of that single decision. Unlike tree-search RL methods (TreeRL, Tree-GRPO), which branch on *general* token entropy and thus mix tool decisions with wording choices, we exploit the binary structure of the call/skip choice, with importance weights correcting the forced first token and a KL penalty keeping the branch gradient inside the same trust region—the region around the current policy within which a gradient step is still reliable—as the main update.

Implementation and headline results. We implement the full pipeline—tool-augmented SFT, a calculator executor, the entropy gate, two-pass forked sampling, and a paired-advantage update folded into a single combined optimizer step—on Qwen 2.5 0.5B for the Countdown arithmetic task. Across four matched conditions (non-tool baseline, tool RLOO, tool RLOO + per-call bonus, and tool RLOO + branching), we find a clear and consistent **null result**: tool use collapses to 0% in every tool condition within 30–70 training steps, and pass@1 lands in the 52–60% band for all of them versus 58% for the tool-less baseline. The branching gate stops firing within the first few steps because, as the tool is pruned, p_{call} saturates near 0 and the binary entropy drops below τ . Neither a +0.05/call reward bonus nor the targeted branching gradient prevents the collapse.

Discussion, limitations, and conclusion. The mechanism behind the null result is instructive rather than a bug. On 3-to-4-number Countdown, Qwen 2.5 0.5B already does the required arithmetic inline most of the time, so a calculator that computes $74 - 7 = 67$ adds no marginal reward; verifier-only RL correctly prunes a tool that does not pay for itself, and the very pruning removes the uncertainty that branching needs to act on. The model is also small enough that it never fully internalizes the tool format. Our contribution is therefore (i) a novel, principled mechanism for targeted credit assignment on binary affordance decisions, implemented and debugged to the point where it is provably inactive rather than harmful (an earlier version caused a $10\times$ KL explosion and a pass@1 crash to 16%, which we diagnosed and fixed), and (ii) a careful, multi-control demonstration of the conditions under which tool-augmented RL is expected to fail. The clear future direction is a harder task paired with a tool that is genuinely useful, where the call/skip decision carries real reward signal and the branching machinery has uncertainty to exploit.

Targeted Counterfactual Branching for Tool-Invocation Decisions in RL Fine-Tuning

Andres Garcia Joshua Hsieh Prakash Koukuntla
CS 224R, Stanford University
{andg, joshhsieh, prakashk}@stanford.edu

Abstract

Knowing when to invoke a tool is a central problem in tool-augmented reasoning, yet trajectory-level RL gives only a diffuse signal on each binary call/skip decision. We introduce Targeted Counterfactual Branching, an inner augmentation to RLOO that, at positions where the policy’s tool-invocation entropy is high, forks the rollout into a forced-call and a forced-skip branch, scores both with the verifier, and forms a paired advantage $A_{\text{call}} = R_{\text{call}} - R_{\text{skip}}$ that isolates exactly that decision. The branch gradient is importance-weighted, KL-constrained, and folded into a single combined optimizer step with the standard RLOO loss. On Countdown with Qwen 2.5 0.5B, we run four matched conditions and find that verifier-only RL drives tool use to 0% in every variant; neither reward shaping nor targeted branching rescues it, and pass@1 is statistically flat (52–60%) across conditions. We trace this to a base model already competent at inline arithmetic on an easy task, making the tool reward-negative and collapsing the very uncertainty the gate depends on.

1. Introduction

Large language models are increasingly trained to act, not just to generate text: they call calculators, run code, and query search engines mid-reasoning [14, 4]. For these models the hard skill is learning at which points in a chain of reasoning does calling the tool actually help, and at which is it wasted effort or even a distraction?

Modern post-training pipelines pair supervised fine-tuning (SFT) with an online RL stage—here REINFORCE Leave-One-Out (RLOO) [1]. The RL stage is the one that can discover behaviors absent from the demonstration data, because the model samples its own rollouts and is rewarded by a verifier. But RLOO assigns a single scalar reward to an entire trajectory. This is a classic credit assignment problem [2]: if a 200-token rollout earns reward 0, every token receives the same diluted gradient, and the specific

decision “should I have called the tool at position t ?” is buried. Worse, the call/skip decision is *binary* and *counterfactual*: the quantity we want is the difference in outcome between calling and skipping, holding everything else fixed. Plain RLOO never constructs that comparison—each rollout commits to one branch of the counterfactual and the other is never observed.

This motivates our central research question:

Can a targeted, counterfactual gradient signal on the binary tool-invocation decision—obtained by forking uncertain rollouts into paired call/skip branches—preserve and improve strategic tool use during RL, beyond what standard RLOO or simple reward shaping achieves?

We study this on the Countdown arithmetic task with Qwen 2.5 0.5B, a clean testbed because a calculator’s effect is well defined and directly visible in the verifier reward. We hypothesized (1) that giving the model a calculator and letting RLOO optimize would improve pass@k; (2) that a per-call reward bonus would preserve tool use; and (3) that targeted branching would preserve and improve tool use more cleanly than reward shaping, by giving dense credit precisely where the call/skip choice is ambiguous.

Our results refute all three hypotheses, and the way they are refuted is the contribution. Verifier-only RL drives tool use to exactly 0% in every tool condition; reward shaping does not stop it and slightly hurts accuracy; and the branching gate stops firing within a few steps because, as tools are pruned, the model becomes certain it should not call, collapsing the entropy the gate needs. Pass@1 is statistically indistinguishable across all four conditions (52–60%). We contribute: (1) a novel branching mechanism that isolates the binary call/skip decision as the unit of a paired counterfactual update, with importance correction; (2) a diagnosis and fix of a branching-induced policy collapse (a $10\times$ KL explosion that cratered pass@1 to 16%); and (3) a controlled, multi-condition characterization of why tool-augmented RL fails when a capable base model meets an easy task.

2. Related Work

Tool-integrated reasoning. Toolformer [14] showed models can learn to call APIs from self-supervised data, and ToRA [4] integrated a code/calculator interpreter into mathematical reasoning. These establish *how* to call tools but largely fix the invocation policy via supervision; we instead ask whether RL can learn *when* to call.

Tree-structured RL for LLM reasoning. A rapid 2025–2026 line of work branches rollouts to obtain intermediate supervision. TreeRL [5] performs on-policy tree search, branching at high *token-entropy* positions and showing tree structure beats chain-style RL on math and code. Tree-GRPO [6] and AT2PO [19] extend branching to multi-turn agentic settings, forking at turn boundaries that include tool calls and computing fork-relative advantages. PORTool [16] organizes rollouts over shared tool-call prefixes and blends fork-relative with trajectory-relative advantages, and CSO [8] identifies critical steps (including tool selection) and verifies them with branch rollouts that swap in expert actions. Closest to us on the entropy axis, AEPO [3] monitors token entropy during agentic rollouts to allocate sampling budget and imposes a branch penalty on consecutive high-entropy tool-call steps. These share a structure—branch, sample k children, compute fork-relative advantages—and a limitation for our purposes: branching is governed by general token entropy (TreeRL, CSO, AEPO), which mixes tool-decision uncertainty with surface-form uncertainty, or by coarse turn/step boundaries (Tree-GRPO, AT2PO), which fire at whole Thought–Action–Observation nodes rather than the call/skip token. Stochastic k -way sampling at a fork is moreover not guaranteed to produce a paired call-vs-skip comparison—both children may make the same tool decision, wasting the signal on the precise axis of interest. AEPO, notably, penalizes high-entropy tool branching for stability, whereas we exploit exactly those positions with a forced call-vs-skip pair.

The binary call/skip decision and reward shaping. A separate line studies *whether* to call a tool directly. “To Call or Not to Call” [17] frames the binary decision via necessity/utility/affordability but learns it with supervised MLP probes on hidden states—not RL counterfactuals. ResT [9] reweights token-level policy gradients for tool-use stability; StepTool [18] and tool-call reward models [10] apply step-grained reward shaping to invocation. These either replace RL with supervised estimators or modify the reward (or its weighting) to encourage proper—and not wasteful—tool use. In our branching trials, we instead leave the verifier reward untouched and add targeted credit assignment. However, we also directly compare our branching-based results against a reward-shaping control.

Stability of extra gradient updates. Adding a second, unconstrained gradient stream to an RL update can violate the trust region and collapse the policy [15, 12]. We encour-

tered exactly this failure mode in an early implementation and resolve it by folding the branch gradient into a single KL-constrained, clipped optimizer step (Sec. 3.8).

Positioning. Our novelty is targeted: to our knowledge, no prior method uses the binary call/skip token as the unit of a *forced, paired* counterfactual RL update. Tree methods fork on general entropy (TreeRL, CSO, AEPO) or whole turn/step nodes (Tree-GRPO, AT2PO); binary-decision work uses supervised probes (To-Call-or-Not) or reward shaping (StepTool, ResT, tool-call RMs). By forcing one branch to call and the other to skip at a high-binary-entropy position, we guarantee a call-vs-skip pair, obtain a low-variance advantage from the shared prefix, and compose cleanly with the standard RLOO loop on that prefix.

3. Method

Our pipeline has four components, the first three standard and the fourth our extension: (i) the calculator tool interface and executor; (ii) tool-less or tool-augmented SFT to obtain a competent starting policy; (iii) the RLOO outer loop with verifier reward; and (iv) the speculative branching inner augmentation. We describe each in turn, then the composition.

3.1. Tool interface and executor

The model invokes a calculator inline using XML-style tags, `<tool>expr</tool><result>value</result>`, where `expr` uses `+` `-` `*` `/` and parentheses. During generation, vLLM [7] stops on `</tool>`; we parse `expr`, evaluate it with the same safe arithmetic evaluator used by the verifier, splice in `<result>value</result>`, and resume generation. We add the four tags as single special tokens so that “open a tool call” is a single, detectable token—essential for the entropy gate (Sec. 3.4). The default Countdown preamble is extended in place with a short tool-affordance description (Fig. 1), and the *same* wording is used at SFT, at RLOO sampling, and at eval so the affordance the model sees is never distribution-shifted.

3.2. Teaching the tool format (Tool-SFT)

Before RL can optimize *when* to call, the model must reliably produce the format. We build a tool-augmented SFT set by regex-wrapping inline arithmetic patterns `expr = N` that already existed in the project SFT data with `<tool>expr</tool><result>N</result>`. Over the resulting $\sim 8.3\text{k}$ tool calls, 97.8% are verified arithmetically correct; the $\sim 2\%$ mismatch is roughly half rounding artifacts and half source-model arithmetic errors (a documented data limitation). The Tool-SFT model emits tools on 100% of responses with ~ 21 calls each before any RLOO.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer.

The Assistant has access to a calculator tool: to use it, the Assistant writes `<tool>EXPRESSION</tool>` where EXPRESSION is an arithmetic expression using +, -, *, /, and parentheses. The system responds with `<result>VALUE</result>` giving the exact result. The Assistant uses the tool for any non-trivial calculation, then bases its reasoning on the result.

Figure 1. The shared preamble used across Tool-SFT, RLOO sampling, and eval. The default Countdown preamble (black) is extended in place with the tool-affordance description (red); the identical wording is used at every stage so the affordance is never distribution-shifted.

3.3. RLOO outer loop

For each prompt we sample a group of k rollouts, score each with the verifier (0.0 no answer tags, 0.1 valid-but-wrong, 1.0 correct), and use the leave-one-out baseline: the advantage of rollout i is $A_i = R_i - \frac{1}{k-1} \sum_{j \neq i} R_j$. The policy-gradient loss is the usual importance-weighted term with an entropy bonus and a KL penalty to a frozen reference. This stage is unchanged by our extension.

3.4. The entropy gate: when to branch

We do not branch everywhere—that is prohibitively expensive and mostly uninformative. At each generated position t , let V_{tool} be the set of tool-opening tokens. From the top- k logprobs vLLM already returns we compute the tool-call mass

$$p_{\text{call}}(t) = \sum_{v \in V_{\text{tool}}} \pi_{\theta}(v \mid x, y_{<t}), \quad (1)$$

where x is the input prompt, $y_{<t} = (y_1, \dots, y_{t-1})$ are the response tokens generated before position t , and $\pi_{\theta}(v \mid x, y_{<t})$ is the policy’s next-token probability for token v . Its binary entropy is

$$H(p_{\text{call}}, 1-p_{\text{call}}) = -[p_{\text{call}} \log p_{\text{call}} + (1-p_{\text{call}}) \log(1-p_{\text{call}})]. \quad (2)$$

A position is a branch candidate iff $H > \tau$ (default $\tau = 0.4$; the maximum is $\ln 2 \approx 0.693$ at $p_{\text{call}} = 0.5$). The intuition: when $p_{\text{call}} \approx 0$ or ≈ 1 the model has already decided, and a counterfactual gives no signal; only near $p_{\text{call}} \approx 0.5$ is the decision genuinely ambiguous. Crucially, we sum mass only over the *exact* single-token form of `<tool>`, never BPE fallbacks like a bare `<`, which would otherwise inflate p_{call} before any `< . . . >` construct. We cap firings at one per rollout for compute.

This is strictly more targeted than branching on total token entropy: a position can have diffuse word-choice entropy yet tiny p_{call} , and such a position is not a meaningful tool decision.

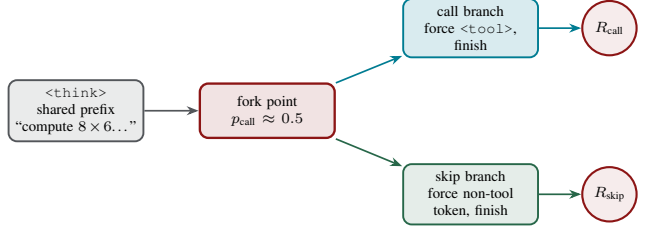


Figure 2. Both branches share the same prefix; only one token differs at the fork. This keeps the reward difference $A_{\text{call}} = R_{\text{call}} - R_{\text{skip}}$ clean, low-variance, and tied to the single call/skip decision.

3.5. Forked sampling (two-pass)

We treat vLLM as a black box and fork via prompt continuation rather than modifying its batched generation internals. **Pass 1** runs normal tool-augmented generation with top- k logprob collection and marks branch points via the gate. **Pass 2**, at each marked point, takes the shared prefix and creates two children: the *call* branch appends the highest-probability tool-opening token; the *skip* branch appends the highest-probability non-tool token. Both then generate normally (with tool execution) to termination and are scored by the verifier. Because we only fork at the gated positions (0–1 per rollout), the overhead is modest.

3.6. Drift-aware importance weights

Forcing the first token biases the estimate, so each branch is reweighted by an importance ratio: the probability of the branch under the distribution we *want* to learn from, divided by the distribution we actually *sampled* from. We use two spans of the same branch continuation. The **suffix** is *all* post-branch tokens—the forced first token plus every token freely generated after it—and is the span the policy gradient acts on. The **free** portion is the suffix *minus* the forced first token, i.e. only the genuinely sampled tokens. The forced token is excluded from the free portion because we chose it deterministically: its probability under the branching sampling distribution is exactly 1, so there is nothing to divide by for it. We likewise distinguish two snapshots of the policy: π_{sample} , the weights at *sampling time* that generated the rollout (the behavior policy, whose logprobs we log), and π_{now} , the *current* weights at the moment of the update, which have drifted slightly because RLOO takes several gradient-accumulation micro-steps between sampling and updating. The clamped importance weight is

$$\text{IS} = \text{clip} \left(\exp \left(\log \pi_{\text{now}}(\text{suffix}) - \log \pi_{\text{sample}}(\text{free}) \right) \right). \quad (3)$$

The denominator covers only the free portion (since the forced token’s sampling probability is 1). At sampling time $\pi_{\text{now}} = \pi_{\text{sample}}$, so the free terms cancel and IS reduces to p_{call} for the call branch and $1 - p_{\text{call}}$ for the skip branch (recovering the static weights of the proposal); under policy drift it additionally tracks how far the policy has moved.

The clip caps a rare low-probability token from producing a giant ratio that would destabilize the update. Executor-injected `<result>` value tokens are masked out of *both* logprobs (the tags themselves stay trainable) so the ratio is computed over the same token set in numerator and denominator.

3.7. Paired counterfactual advantage and loss

With branch rewards $R_{\text{call}}, R_{\text{skip}}$, the leave-one-out advantages of a two-element group reduce exactly to

$$A_{\text{call}} = R_{\text{call}} - R_{\text{skip}}, \quad A_{\text{skip}} = -A_{\text{call}}. \quad (4)$$

Unlike RLOO’s baseline—a comparison against the *average* of other full trajectories—this compares two trajectories identical except for one token, eliminating prompt and prefix confounds and yielding a much lower-variance estimate of that decision’s value. The per-pair loss, applied *only* to post-branch (suffix) tokens to avoid double-counting the prefix already covered by RLOO, is

$$\begin{aligned} L_{\text{pair}} = & -[\text{IS}_c A_{\text{call}} \log \pi(\text{call}) \\ & + \text{IS}_s A_{\text{skip}} \log \pi(\text{skip})] \\ & + \lambda_{\text{KL}}(\text{KL}_c + \text{KL}_s), \end{aligned} \quad (5)$$

where each branch forward also runs the reference model to add a per-token KL term, so the branch stream lives in the *same* trust region as the main update.

3.8. Composition with RLOO: one combined step

$$L_{\text{total}} = L_{\text{RLOO}} + \frac{\text{branch_coef}}{N_{\text{branch}}} \sum_{\text{pairs}} L_{\text{pair}}. \quad (6)$$

The $1/N_{\text{branch}}$ normalization keeps the total branch gradient scale constant regardless of how many branches fire. Critically, the branch backward *accumulates onto the same gradient buffer* as the RLOO backward; we then take a *single step* $\rightarrow \text{clip} \rightarrow \text{scheduler} \rightarrow \text{zero_grad}$. One backward-accumulation, one clip, one optimizer step—so the same KL trust region and gradient clip cover both streams. This single-step design is the fix for the policy collapse described in Sec. 5.2; the original two-step version (a second, unclipped optimizer step for branching) violated the trust region.

4. Experimental Setup

Task and model. Countdown: given a target and a set of numbers, produce an arithmetic equation using exactly those numbers that hits the target, wrapped in `<answer>...</answer>`. We use 3-to-4 number problems from `asingh15/countdown_tasks_3to4`, the Countdown reasoning benchmark popularized by

Algorithm 1 One RLOO step with targeted branching

- 1: Sample group of k rollouts per prompt (Pass 1), collect top- k logprobs
- 2: Compute LOO advantages A_i ; accumulate L_{RLOO} backward (*defer* step)
- 3: **for** each rollout **do**
- 4: $t^* \leftarrow \arg \max_t H(p_{\text{call}}(t), 1 - p_{\text{call}}(t))$ with $H > \tau$
- 5: **if** t^* exists **then** ▷ at most one per rollout
- 6: Fork at t^* : **call** (force `<tool>`), **skip** (force top non-tool)
- 7: Generate & score both (Pass 2) $\rightarrow R_{\text{call}}, R_{\text{skip}}$
- 8: $A_{\text{call}} \leftarrow R_{\text{call}} - R_{\text{skip}}$; compute IS, KL on suffixes
- 9: Accumulate $\frac{\text{branch_coef}}{N_{\text{branch}}} L_{\text{pair}}$ backward
- 10: **end if**
- 11: **end for**
- 12: **Single** update: clip $\rightarrow$ optimizer.step $\rightarrow$ scheduler $\rightarrow$ zero_grad

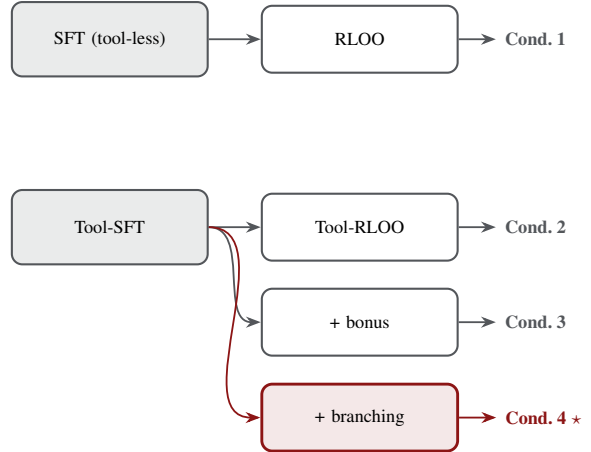


Figure 3. The four matched conditions. A tool-less SFT→RLOO baseline (Cond. 1) plus three conditions sharing a Tool-SFT init: plain tool RLOO (Cond. 2), tool RLOO with a +0.05/call bonus (Cond. 3), and tool RLOO with our branching extension (Cond. 4*). Cond. 4 adds no bonus; only the toggles between rows differ.

TinyZero [11]. The base model is Qwen 2.5 0.5B [13] throughout; per project rules we implement all training loops manually (no high-level RL trainers).

Datasets. SFT: `Asap7772/cog_behav_all_strategies` (tool-less) and `joshsieh/countdown-tool-sft` (our tool-augmented derivative). RLOO and eval prompts: `asingh15/countdown_tasks_3to4`. For all tool conditions we augment every RLOO and eval prompt with the tool-affordance preamble of Fig. 1—the same wording used at Tool-SFT—so the calculator is advertised identically at training and test time; the tool-less baseline (Cond. 1) sees the unmodified preamble.

Conditions. Four matched conditions isolate each com-

k	SFT	RLOO	Δ
1	32.5 $\pm$ 6.6	58.0 $\pm$ 7.0	+25.5
2	47.3 $\pm$ 7.1	66.9 $\pm$ 6.7	+19.6
4	60.2 $\pm$ 6.9	72.7 $\pm$ 6.3	+12.5
8	70.7 $\pm$ 6.4	76.2 $\pm$ 6.0	+ 5.5
16	80.0 $\pm$ 5.7	78.0 $\pm$ 5.9	- 2.0

Table 1. Pass@ k (%) for the tool-less pipeline; $\pm$ is binomial SE ($n=50$ prompts). Pass@1 gain is ~ 2.7 joint SEs, high- k change is within noise.

ponent (Fig. 3 shows the pipeline; Fig. 2 the branching mechanism): (1) non-tool SFT $\rightarrow$ RLOO (baseline); (2) Tool-SFT $\rightarrow$ tool RLOO, no bonus; (3) tool RLOO + a +0.05/correct-call reward bonus (reward-shaping control, capped at 3 calls); (4) tool RLOO + branching, no bonus. Conditions 2–4 share the same Tool-SFT init and uncapped tool rounds (context length is the cap). The key comparisons are 1-vs-2 (does the tool help at all?), 2-vs-3 (does reward shaping preserve tool use?), and 2-vs-4 (does branching preserve/improve it?). The *central* comparison is 3-vs-4: it pits the two mechanisms for keeping tools alive against each other—broad reward shaping (a +0.05/call bonus, Cond. 3) versus a targeted gradient at the uncertain call/skip decision (branching, Cond. 4)—under an otherwise identical pipeline. Cond. 3 modifies the reward and assigns credit at the trajectory level; Cond. 4 leaves the reward untouched and assigns credit at the specific decision.

Hyperparameters. All RLOO runs: $\text{lr} = 1\text{e-}5$, batch size 128, group size 8, KL coef 0.001, entropy coef 0.001, 100 steps, constant schedule. Branching: $\tau = 0.4$, max 1 branch/rollout, $\text{branch_coef} = 0.5$, gradient clip 1.0 (a real trust region over the combined step). SFT: $\text{lr} = 5\text{e-}5$, batch 64, 6 epochs, cosine. Training on H100 GPUs via Modal.

Metrics. Primary: $\text{pass}@k$ ($k \in \{1, 16\}$) on 50 test prompts $\times$ 16 responses, with the tool preamble and executor enabled for tool conditions. Secondary, logged over training: fraction of responses containing `<tool>`, mean tool calls per response, and the number of gate-eligible branching positions per rollout. We chose $\text{pass}@1$ as the headline reliability metric and $\text{pass}@16$ as a diversity/coverage metric; the tool-fraction trajectory is the direct probe of our hypotheses about tool preservation.

5. Results

5.1. Quantitative results

The pipeline works; the tool does not help. Table 1 confirms the core pipeline: tool-less SFT $\rightarrow$ RLOO lifts $\text{pass}@1$ from 32.5% to 58.0% (+25.5pp), clearing the project’s 0.45 RLOO threshold. With $n=50$ prompts, each Pass@1 estimate has a binomial SE of roughly $\pm 7\%$, so the 52–60% spread across conditions is well within one standard error and we cannot distinguish them.

	C1	C2	C3	C4
Pass@1	58.0 $\pm$ 7.0	60.0 $\pm$ 6.9	54.0 $\pm$ 7.0	52.0 $\pm$ 7.1
Pass@16	78.0 $\pm$ 5.9	74.0 $\pm$ 6.2	64.0 $\pm$ 6.8	70.0 $\pm$ 6.5
Tool%	n/a	0	0	0

Table 2. Best-checkpoint evals (50 prompts $\times$ 16 responses). **C1:** non-tool baseline; **C2:** Tool RLOO; **C3:** Tool + 0.05/call bonus; **C4:** Tool + branching. $\pm$ is binomial SE, $\sqrt{\hat{p}(1-\hat{p})/n}$, $n=50$. All Pass@1 Wilson 95% CIs overlap (range 38.5–72.4%); two-proportion z -tests on 3-vs-4 ($z=0.20$) and 1-vs-4 ($z=-0.60$) cannot reject equality.

Table 2 is the headline comparison. All three tool conditions land within noise of the tool-less baseline on $\text{pass}@1$ (52–60% vs. 58%), and *every* tool condition ends training with 0% tool use. The tool added no net capability. The reward bonus (Cond. 3) actually *hurts*: 54% $\text{pass}@1$ and 64% $\text{pass}@16$, the worst of the set, consistent with a competing gradient that the verifier reward must undo. On the central 3-vs-4 comparison, *neither* mechanism preserves tool use—both reach 0%—so on this task the value of a tool call is recoverable neither broadly (bonus) nor at specific decisions (branching). Between the two, branching is the gentler intervention: it matches the bonus on $\text{pass}@1$ (52 vs. 54%, within noise) but retains more $\text{pass}@16$ diversity (70 vs. 64%) without distorting the verifier reward, whereas the bonus injects a competing gradient and lands worst on both axes.

Tool use collapses early and completely. Fig. 4 shows when the tool collapses. From a Tool-SFT init that uses tools on 100% of responses (~ 21 calls each), all three tool conditions collapse to near-0% within 30 steps and exactly 0% by step 70. The collapse is front-loaded: the model first stops doing *multiple* calls per response, then drops the format entirely. The bonus *doubles* initial tool usage (41.9% vs. 21.3% at step 10) but cannot sustain it—by step 30 it is below 1%. Branching collapses tool use *fastest* (50% $\rightarrow$ 4% in 20 steps): the targeted gradient fires on exactly the uncertain call/skip positions, where the verifier favors skip (the inline path is already correct), so it *accelerates* the same pruning rather than countering it.

The branching gate stops firing. Fig. 5 shows the number of gate-eligible branching positions per rollout over training. It drops to near zero within the first few steps. For every method, the number of potential branch positions reaches zero for all $\tau \geq 0.2$. The mechanism is direct: as the tool is pruned, p_{call} saturates near 0 everywhere, so the binary entropy stays below τ and the gate never fires. Branching is thus inapplicable in this regime—not because the machinery is broken, but because there is no uncertainty left to optimize. The branch loss is structurally 0 after the first few steps, and Cond. 4 converges to the same terminal regime as Cond. 2.

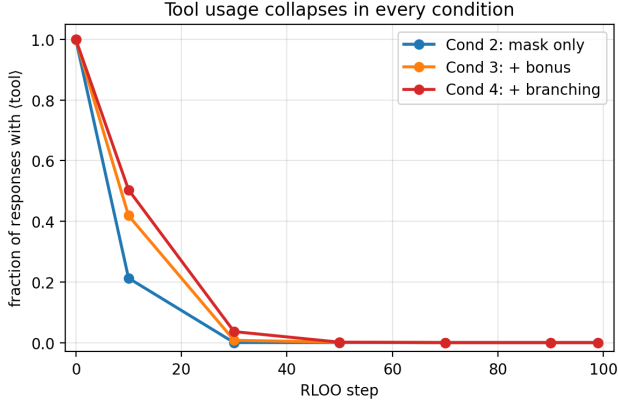


Figure 4. Fraction of responses containing `<tool>` across training for Conditions 2–4. All three collapse to 0% by steps 30–70, with the branching condition pruning fastest.

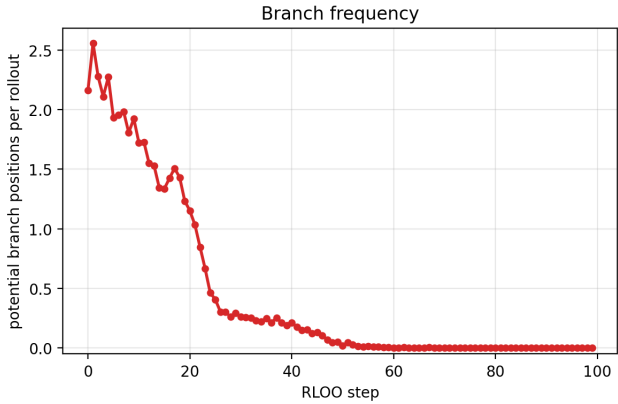


Figure 5. Potential branching positions per rollout across RLOO steps. The gate stops firing within the first few steps: p_{call} saturates near 0, so binary entropy falls below $\tau = 0.4$.

5.2. Qualitative analysis

Why non-tool rollouts win the group. The base model is already good at 3-to-4-number arithmetic. We extract every inline arithmetic step the model writes in its own reasoning—each statement of the form “number op number = result”, e.g. $44 + 19 = 63$ or $74 - 7 = 67$ —and check it against the true value: 91.1% are correct (4,362/4,786 steps). The model already computes the vast majority of these correctly on its own, so grounding them in a calculator earns no extra reward. Within a group of 8 rollouts, the tool-using ones are *more* likely to be malformed (tag swaps, missing `</tool>`, context blowups from many calls) and so fall into the wrong bucket more often. The LOO baseline then assigns positive advantage to the non-tool rollouts and negative advantage to the tool-using ones; the policy gradient pushes tool-opening tokens down and inline-arithmetic tokens up. This is the textbook RL outcome of pruning a behavior that does not pay for it-

Shared prefix (both groups reason identically):
`<think> ...get close to 17:  $73 - 49 = 24$  (too high); work backwards from 17 (odd)...`
Call (12/16): `... $73 - 41 = 32$, then <tool> $32 - 49$ </tool><result>-17</result> details to a negative value, no valid answer.
 mean reward 0.32
Skip (4/16): stays inline and reaches a correct equation for 17. mean reward 0.78
 $A_{\text{call}} = R_{\text{call}} - R_{\text{skip}} \approx 0.32 - 0.78 = -0.46$`

Figure 6. A real call/skip split (prompt: target 17 from [49, 41, 73]; Tool-RLOO, step 10). Across the 16 sampled rollouts the policy divides at one reasoning juncture—the kind of high- p_{call} position the entropy gate targets. The skip (inline) group outscores the call (tool) group, giving a negative empirical advantage $A_{\text{call}} \approx -0.46$, so the branch gradient pushes the tool token down. (Per-pair branch text was not logged; this reconstructs the decision from the same-prompt rollouts.)

self.

A call/skip decision in practice. Per-pair branch text was not retained during training, but the decision the gate targets is visible directly in the rollouts. Fig. 6 takes one prompt (target 17 from {49, 41, 73}) at step 10, where the policy genuinely splits: 12 of the 16 sampled rollouts open the calculator at the same reasoning juncture and 4 stay inline. The two groups share a near-identical prefix and diverge at one decision—exactly the high- p_{call} position the entropy gate fires on. The inline (skip) group earns mean reward 0.78; the tool (call) group earns only 0.32, an empirical paired advantage $A_{\text{call}} \approx -0.46$. Because the verifier favors skip, the branch gradient at this position pushes the tool token *down*—a microcosm of why the branching condition prunes tool use fastest (Fig. 4). Even where the model is most uncertain about calling, calling is the worse choice.

The tool scaffold is brittle. Beyond being unhelpful, tool use is actively risky for the small model. At step 10 of tool RLOO, tool-using responses average 5.2 calls (max 12); 181/579 contain ≥ 8 calls and 138 never emit an `<answer>` at all, having exhausted the 2,048-token sampling budget *we* set mid-reasoning, and 23% have mismatched tag counts. Fig. 7 shows a representative failure: the model spirals through repeated calls, *contradicts its own tool output* (the executor returns `<result>50</result>` for $39 + 11$, yet the next line asserts $39 + 11 = 40$), and is finally truncated mid-call with no closing `</result>` and no answer, scoring 0. A rollout like this drags the whole group’s tool-using branch into the wrong bucket, reinforcing the prune. The 2,048-token cap is our configuration choice, far below Qwen 2.5’s 32K native context, and we did not sweep it—so we cannot say how much of this truncation a larger budget would remove. Plausibly much of it: the longest rollout we observed used only 12 calls, so a few thousand more tokens


```

... 95 - 56 = 39
<tool>39 + 11</tool><result>50</result>
...
Final sequence:
<tool>95 - 56</tool><result>39</result>
39 + 11 = 40 (ignores the tool result 50; also wrong)
40 - 2 = 38
Aha! Let's try:
<tool>95 - 56</tool> (truncated mid-call: no result, no
answer ⇒ score 0)

```

Figure 7. A representative malformed tool-using rollout (Cond. 2, step 10). The model runs away with repeated calls, overrides its own tool result with incorrect inline arithmetic, and is cut off mid-call before producing an answer. Truncation occurs when runaway tool calls exhaust the 2,048-token sampling budget (our setting, not a model limit) before an answer is produced.

might let most runaway rollouts finish. We therefore treat truncation as a *contributing symptom*, not the root cause. Even with unlimited budget the underlying economics are unchanged—inline arithmetic is already 91% correct, so a tool call that merely reproduces it stays reward-neutral, and tool use collapses across all three control runs regardless. A budget sweep is a concrete robustness check we leave to future work.

The +0.05 bonus is too small to matter. A +0.05/call bonus on correct rollouts may not be enough to motivate the model to learn to use the tool properly. It visibly shifts step-10 behavior (doubling tool usage) but is overwhelmed by step 30. Pushing the bonus higher could preserve tool use if it heavily rewards the rollouts that do get tool use right.

A diagnosed and fixed policy collapse. Our first branching implementation followed our original proposal literally: a second, separate optimizer step for the branch gradient. With 200–800 branch pairs per step, each contributing an unclipped backward, this doubled the per-iteration update magnitude with no trust region. KL exploded $10\times$ ($0.1 \rightarrow 1.0$), reward degraded, and pass@1 cratered to **16%** (pass@16 32%)—far below even the SFT init. We diagnose this as a trust-region violation [15, 12] and fix it with the four changes in Sec. 3.8: a single combined optimizer step, $1/N_{\text{branch}}$ scaling, IS correction on the branch suffixes, and a KL term on the branch stream. Post-fix, KL stays bounded for the whole run and pass@1 recovers to the 52% band—i.e. the branching machinery is no longer harmful, merely inactive. This is itself a useful result: it confirms the earlier degradation was an implementation bug, not evidence against the method.

Ruling out the masking confound. An early tool-RLOO run included executor-injected `<result>` tokens in the policy loss, creating a sample/policy logprob mismatch. We re-ran a masking-only control: with masking correctly applied, tool use collapses *faster* (21%→0% by step 30), not slower, and final pass@1 is identical (60%). The collapse is

therefore structural, not an artifact of the masking bug—one of three convergent controls (masking, bonus, branching) that all reach 0% tool use.

6. Discussion

Limitations. Two are load-bearing. First, the *task is too easy*: 3-to-4-number Countdown is solvable inline by the base model, so the calculator is reward-negative and there is no incentive to keep it. Second, the *model is small*: Qwen 2.5 0.5B never fully internalizes the `<tool>/<result>` syntax even after Tool-SFT, so RL inherits an unreliable scaffold. Together these make tool abandonment the *correct* optimization outcome, which in turn collapses the entropy our gate needs—a coupling that no amount of branching tuning can overcome on this task. Our eval is also modest (50 prompts). Finally, the 2,048-token sampling budget is a confound we did not control for: it truncates a meaningful fraction of runaway tool-using rollouts before they answer (Fig. 7), inflating how often tool use scores 0. We argue this is a contributing symptom rather than the root cause—inline arithmetic is already 91% correct, so the tool stays reward-neutral even when rollouts finish—but a budget sweep at Qwen 2.5’s full 32K context is the cleanest way to confirm it, and we leave it to future work.

A stronger reward as a lever. The +0.05/call bonus in Cond. 3 was deliberately gentle—about a 5% nudge to a correct rollout’s reward—and it was overwhelmed within 30 steps. But its *transient* effect is suggestive: it doubled step-10 tool usage (42% vs. 21%) before the verifier gradient reasserted itself. A larger or better-shaped reward might cross the threshold where tool use becomes worth keeping: e.g. a bonus that scales with the number of *load-bearing* calls, an explicit penalty on malformed calls (the dominant tool-failure mode, Fig. 7), or a curriculum that protects tool use early and anneals the bonus as the format stabilizes. The plausible mechanism is that a sufficiently strong, correctness-gated reward stops the model from abandoning the scaffold long enough to actually *internalize* the `<tool>/<result>` format—turning a brittle, occasionally malformed behavior into a reliable one—after which the tool could pay for itself even on this task. The cost is reward distortion: too large a bonus trains the model to call the tool decoratively, so the open question is whether the preserved tool use is load-bearing or merely farmed. This is the most direct follow-up our results point to, alongside the harder-task direction below.

7. Conclusion

We set out to give RL a targeted, counterfactual signal on the binary decision of *when* to call a tool, and we built the full mechanism: an entropy gate on tool-call mass, forked

call/skip sampling, drift-aware importance weights, and a paired advantage folded into a single KL-constrained optimizer step. On Countdown with Qwen 2.5 0.5B the honest finding is a null result with a clear cause: a capable base model on an easy task makes the calculator reward-negative, verifier-only RL correctly prunes it in every variant, and the pruning erases the uncertainty branching depends on. The take-home is that *targeted credit assignment cannot manufacture a learning signal where the task provides none*—the tool must actually pay for itself before “when to call it” becomes a question worth optimizing. The concrete next step is to pair the method with a harder task and a genuinely useful tool (e.g. an oracle-like solver, or larger Countdown / multi-step word problems where inline arithmetic is unreliable), where p_{call} stays ambiguous and the paired counterfactual has real reward to separate. Scaling the base model so the tool format is reliable is the complementary direction. The mechanism is built, debugged, and ready for that regime.

8. Team Contributions

All three authors contributed to the experimental design, analysis, and writing.

Andres Garcia: Extension planning and proposal write up. RLOO extension infrastructure (tool-sampling worker, entropy gate, paired advantage update). Built tool-augmented data pipeline. Trained and evaluated Tool RLOO, Tool + bonus. Contributed to poster and paper.

Joshua Hsieh: Base RLOO implementation. Combined step trust-region fix. Trained and evaluated base SFT, IPO, RLOO, Branching RLOO. Contributed to poster and paper.

Prakash Koukuntla: Base SFT and IPO implementations. Aided and facilitated training and evaluation. Contributed to poster and paper.

Changes from the proposal. (1) We *added a fourth condition*, a +0.05/call reward-shaping control, not in the original three-condition plan, to disentangle reward shaping from targeted gradient as mechanisms for preserving tool use. (2) The proposal described branching as a separate gradient update; our initial implementation of this caused a trust-region violation and policy collapse, so we *changed the composition to a single combined optimizer step* with IS correction and a KL term on the branch stream (Sec. 3.8). (3) We *added spliced-result-token masking* to keep importance ratios honest, a detail not anticipated in the proposal. (4) The proposed secondary metrics (call-decision calibration, convergence-speed curves) could not be meaningfully reported because the branching gate stopped firing within a few steps; we *substituted* tool-use-collapse and branch-frequency trajectories, which are the informative metrics in this regime. (5) The illustrative tool tag changed from `<python>` to a `<tool>/<result>` calculator suited to arithmetic.

References

- [1] A. Ahmadian, C. Cremer, M. Gallé, M. Fadaee, J. Kreutzer, O. Pietquin, A. Üstün, and S. Hooker. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [2] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [3] G. Dong, L. Bao, Z. Wang, K. Zhao, X. Li, J. Jin, J. Yang, H. Mao, F. Zhang, K. Gai, G. Zhou, Y. Zhu, J.-R. Wen, and Z. Dou. Agentic entropy-balanced policy optimization. *arXiv preprint arXiv:2510.14545*, 2025.
- [4] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [5] Z. Hou, Z. Hu, Y. Li, R. Lu, J. Tang, and Y. Dong. TreeRL: LLM reinforcement learning with on-policy tree search. *arXiv preprint arXiv:2506.11902*, 2025.
- [6] Y. Ji, Z. Ma, Y. Wang, G. Chen, X. Chu, and L. Wu. Tree search for LLM agent reinforcement learning (tree-grpo). *arXiv preprint arXiv:2509.21240*, 2025.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with paged attention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [8] M. Li, Q. Zeng, T. Fang, Z. Liang, L. Song, Q. Liu, H. Mi, and D. Yu. Verified critical step optimization for LLM agents. *arXiv preprint arXiv:2602.03412*, 2026.
- [9] Z. Lin, X. Wang, J. Cao, J. Chai, G. Yin, W. Lin, and R. He. ResT: Reshaping token-level policy gradients for tool-use large language models. *arXiv preprint arXiv:2509.21826*, 2025.
- [10] D. Ma, Z. Yang, H. Xu, H. Fang, K. Yu, and L. Chen. Empowering LLM tool invocation with a tool-call reward model. In *OpenReview preprint (id LnBEASInVr)*, 2025.
- [11] J. Pan, J. Zhang, X. Wang, et al. TinyZero: Reproducing the countdown and multiplication reasoning of deepseek r1-zero. <https://github.com/Jiayi-Pan/TinyZero>, 2025.
- [12] P. Qi, X. Zhou, Z. Liu, T. Pang, C. Du, M. Lin, and W. S. Lee. Rethinking the trust region in LLM reinforcement learning. *arXiv preprint arXiv:2602.04879*, 2026.
- [13] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [15] Y. Tang and R. Munos. On a few pitfalls in KL divergence gradient estimation for RL. *arXiv preprint arXiv:2506.09477*, 2025.

- [16] F. Wu, W. Zhu, Y. Zhang, S. Chatterjee, J. Zhu, F. Mo, R. Luo, and J. Gao. PORTool: Importance-aware policy optimization with rewarded tree for multi-tool-integrated reasoning. *arXiv preprint arXiv:2510.26020*, 2025.
- [17] Q. Wu, S. Das, M. Amani, A. Nag, S. Lee, K. P. Gummadi, A. Ravichander, and M. B. Zafar. To call or not to call: A framework to assess and optimize LLM tool calling. *arXiv preprint arXiv:2605.00737*, 2026.
- [18] Y. Yu, Z. Wang, W. Ma, S. Wang, C. Wu, Z. Guo, and M. Zhang. StepTool: Enhancing multi-step tool usage in LLMs via step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.
- [19] Z. Zong, D. Chen, Y. Li, Q. Yi, B. Zhou, C. Li, B. Qian, P. Chen, and J. Jiang. AT²PO: Agentic turn-based policy optimization via tree search. *arXiv preprint arXiv:2601.04767*, 2026.